# Ellis Trust and Risk Review

## How an AI agent was deployed into a live production platform without the power to harm it, and what it found that could.

**As of** 10 Nov 2026 **Prepared by** Quincy Nkwain Ninying, Forward Deployed Engineer **Accountable** K.D. **Governance** A.M.

**Scope** 40 verified capabilities and 16 sandbox rungs; platform code to `456de02fd` (open risks re-checked 10 Nov)

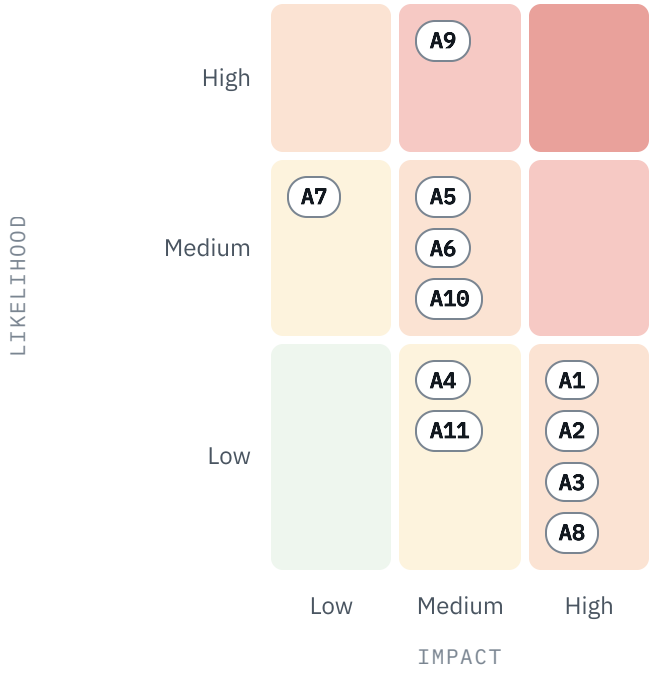
| 0 | 10 | 11 | 25 | 11 |
|---|---|---|---|---|
| ways for Ellis to write to the live platform | hard limits that hold at every autonomy level | risks from the agent, each with a named control | live risks Ellis found in the platform | of those 25 already fixed upstream |

## PART 1 · RISK FROM THE AGENT

HEAT MAP

### No risk from Ellis sits in the high-likelihood, high-impact corner

Each risk is placed after its control. The most likely one is the honest one: Ellis has not yet run on live records, so real data may surprise it.




**How to read it.** Rows are how likely the risk is once its control is in place; columns are how much harm it would do. Darker cells need more attention.

**The high-impact column is all low likelihood.** Writing to the platform, a false green, a leak and self-promotion are each blocked by construction or by a test, not by good intentions.

**The medium-impact, medium-likelihood risks are about trust and judgement:** a finding that overstates, a rule copied from platform code that drifts as the code moves, and a proposal followed without checking.

CONTROL REGISTER

## Every risk from the agent has a control you can point to

| # | RISK | CONTROL | RESIDUAL |
|---|---|---|---|
| **A1** | Ellis changes the live platform | Capabilities receive a read-only source with no write method; repairs run only on a sandbox copy; there is no production target in the code `src/readSource.ts, src/sandbox/sandboxStore.ts` | Low |
| **A2** | Ellis reports healthy when it cannot see (a false green) | Three answers, with unknown above clean; each capability's acceptance test removes the signal and fails if the answer is not unknown; key rules are broken on purpose to prove a test catches it `no-evidence-is-not-health.md; each capability's tests` | Low |
| **A3** | Ellis leaks a secret or personal data | Access snapshots with password, token or secret columns are refused unread; security findings pass a field whitelist; artifacts are scanned before they are published `src/access/accessSnapshot.ts, src/capabilities/securityPostureScan.ts` | Low |
| **A4** | A baseline test reaches outside its sandbox | Tests run under Node's permission model: their own folder only, no other programs, no network; a refused write is never accepted as a result; file names are checked `src/sandbox/baselineRunner.ts` | Low |
| **A5** | A finding overstates the problem and costs the owners' trust | Check the platform's tests, baselines and documents first; label each finding live, latent or reporting gap; A.M. reviewed line by line until 30 Oct, when defects moved to Ellis's own output. Of 76 reviewed, 9 were rewritten `colaberry-platform-reliability-audit.pdf` | Medium |
| **A6** | A rule Ellis copied from platform code drifts as the code changes | Every copied rule names its commit and file and line; citations are rechecked against current code when a capability touches them. On 10 Nov all 18 commands that read the platform were re-run against `456de02fd`, 368 commits on; none failed | Medium |
| **A7** | An export is incomplete or malformed | Malformed input is refused with exit code 1; anything missing is reported as unknown or not recorded, never filled in | Low |
| **A8** | Ellis raises its own authority | Nothing in Ellis can change an autonomy level or a policy; K.D. approves each step; 23 of 48 decisions are reserved to a person at every level `autonomyLadder.ts:213-219` | Low |
| **A9** | Real records behave differently from the tests and examples | Every answer cites its rows, so a wrong one can be traced; the first live check compares one export with Ellis's answers, with K.D. | Medium |
| **A10** | A person follows a proposal to replay or abandon without checking it | Each proposal carries its evidence and row ids; replay is labelled unproven on every answer; a person decides at every level `src/capabilities/deadLetterTriage.ts` | Medium |
| **A11** | A sandbox operation writes outside its own folder | Each rehearsal gets a fresh folder, proven empty; a restore target is proven inside it before anything is restored; a backup carrying a statement that could reach another file is refused unrun; the R3 operations need a named operator. The refusal is shown to matter: with it removed on purpose in testing, a backup did write a stray file outside its folder, and a test caught it `src/sandbox/sandboxRestore.ts, src/sandbox/schemaDatabases.ts` | Low |

LIVE RISKS BY AREA

## 25 live risks across five areas; 11 are already fixed

From the Colaberry Platform Reliability Audit. Each cell counts the area's live risks by status. Re-checked on 10 Nov against 368 platform commits since 5 Nov: none of the 14 open risks changed. The files they cite are unchanged, or changed only for unrelated work.

| AREA | FIXED | CONFIRMED | DEFERRED | AWAITING | TOTAL |
|---|---|---|---|---|---|
| **Security and access**<br>a staff role raised to owner; a secret kept in its ticket; critical alerts never ticketed (E6, E10, E8, E11, 8) | 2 | 1 | 2 | 0 | 5 |
| **Control and containment**<br>the grant map is not enforced; a pause is still undone when a run fails (2, 25, 32, 33, 58, E3, E4, E5) | 5 | 2 | 1 | 0 | 8 |
| **Records that say it happened**<br>a fix announced but never applied; tickets closed as recovered on no evidence (E1, 13, 19, 20, 49, 61) | 2 | 4 | 0 | 0 | 6 |
| **Data and resources**<br>decision history deleted before it was archived; disk never freed (9, 10, 15) | 0 | 3 | 0 | 0 | 3 |
| **People on the receiving end**<br>the same email twice, to the lead who had just replied; a weekly report that never arrives (16, 17, 57) | 2 | 1 | 0 | 0 | 3 |

Fixed upstream · Confirmed, open · Deferred, waiting on a decision · Awaiting review

Cell shade grows with the count

HOW TRUST IS EARNED

## Four kinds of evidence, each checkable by someone else

**40**

**Verified by the platform**

Each capability is confirmed by the platform reading this repository, not by a self-report.

**61 / 76**

**Held under review**

Findings A.M. checked line by line that held as written, or turned out worse.

**5**

**Public fixes**

Platform pull requests 3410, 3422, 3430, 3435 and 3439 fixed fourteen findings.

**926**

**Tests that catch defects**

Key rules are broken on purpose, one at a time, and a named test is shown to fail each time.

RESIDUAL RISK

## What is not yet covered, and the next control for each

| RESIDUAL | NEXT CONTROL |
| --- | --- |
| **Nothing has run on live records**<br>All evidence so far is tests, examples and real platform code, not real platform data. | One export compared with Ellis's answers, together with K.D., before any capability moves up a level. |
| **The sandbox is a seat belt, not a security boundary**<br>Node's permission model stops trusted code making mistakes; its own documentation says it does not protect against malicious code. | Baselines run only test code Ellis's engineer wrote; anything untrusted would need a container or a separate machine. |
| **K.D. cannot yet open the agent's own pages**<br>K.D. has no admin_users row; the portal mints an admin login from K.D.'s management role (A.M., 4 Nov). The agent's pages also need the domain record to carry K.D.'s work address. | A.M. is correcting the domain record; a sourced written briefing until then. |
| **Copied platform rules can go stale**<br>The platform changes daily; a rule Ellis copied may no longer match. | Last re-checked 10 Nov against `456de02fd`, with nothing stale. Re-check again before each capability's first live run. |

SOURCES

- Ellis AI Employee Charter: the hard limits and decision rights behind Part 1
- Colaberry Platform Reliability Audit: the 25 live risks behind Part 2
- Ellis source: every control named in the register
- Node.js permission model: what it restricts, and that it is not a security boundary against malicious code